

CoProof — Speech de Defensa de Tesis

Proyecto Integrador II · CETI · Mayo 2026

Duración estimada por slide: 30–55 segundos. **Duración total estimada:** ~10–12 minutos. Las pausas naturales entre slides están indicadas con [pausa].

Slide 1 — Portada

Buenas tardes. Somos David López, Daniel Tejeda y Emiliano Flores, y hoy les presentamos **CoProof**: una plataforma colaborativa de demostración formal de teoremas, desarrollada como proyecto integrador del programa de Ingeniería en Desarrollo de Software del CETI.

Lo que van a ver a continuación no es solo un sistema de software. Es una respuesta a un problema real que ocurrió frente a los ojos de toda la comunidad matemática mundial, hace menos de diez años.

[pausa]

Slide 2 — Sir Michael Atiyah y la Hipótesis de Riemann

En septiembre de 2018, uno de los matemáticos más importantes del siglo veinte —Sir Michael Atiyah, ganador de la Medalla Fields y del Premio Abel— anunció que tenía una demostración de la **Hipótesis de Riemann**, uno de los siete Problemas del Milenio, sin resolver durante ciento sesenta años.

La comunidad matemática reaccionó de inmediato: foros, publicaciones, anotaciones, contraargumentos. Atiyah falleció en enero de 2019. Meses después el consenso fue unánime: la prueba contenía un **error lógico fatal**.

Pero llegar a ese consenso tomó **meses de colaboración asíncrona**. Meses de debate, de incertidumbre, de preguntas sin árbitro neutral. Eso nos llevó a preguntarnos: ¿tiene que ser así?

[pausa]

Slide 3 — El Problema de Fondo

El problema de fondo no es que Atiyah se haya equivocado. Los matemáticos se equivocan; es parte del proceso. El problema es **cuánto tiempo tomó saberlo con certeza**.

La revisión de pruebas matemáticas es lenta, asíncrona y, sobre todo, **subjativa**. No existe un estándar verificable por máquina para decir "este paso es válido". Cuando se distribuye la revisión entre varios colaboradores, los desacuerdos se multiplican sin un árbitro neutral.

Entonces la pregunta que guía este proyecto es: ¿Qué pasaría si la corrección lógica fuera tan inequívoca como **un error de compilador**?

[pausa]

Slide 4 — Corrección Lógica: de Matemáticas a Lógica Pura

Para responder esa pregunta, necesitamos entender qué significa verificar una demostración.

Tomemos el ejemplo clásico: la suma de los primeros n naturales es $n(n+1)/2$. La demostración por inducción tiene una **estructura lógica pura**: primero se verifica el caso base, luego se establece el paso inductivo, y de esas dos premisas se deduce la conclusión para todo n .

Esa estructura — $P \rightarrow Q$ por lo tanto para todo n — es exactamente un **modus ponens**. La corrección de la prueba no depende de intuición ni de experiencia; depende de que cada paso siga reglas formales admitidas.

Si podemos representar esas reglas en software, tenemos un árbitro infalible.

[pausa]

Slide 5 — Las Pruebas son Grafos, no Listas

Hay algo más que debemos entender sobre la estructura de una demostración matemática: **no es lineal**.

Pensemos en cómo se prepara una hamburguesa. Para ensamblarla necesitamos pan tostado, carne asada y tomate picado. Esos tres pasos son **completamente independientes entre sí** y pueden hacerse en paralelo.

Una prueba matemática funciona igual. El teorema raíz depende de lemas, y esos lemas dependen de casos base. Los lemas que no se relacionan entre sí pueden resolverse de forma **paralela**, asignados a distintos colaboradores.

Esta estructura es un **grafo acíclico dirigido**, un DAG. Y es exactamente el modelo que CoProof usa para organizar el trabajo colaborativo.

[pausa]

Slide 6 — Automatizando la Formalización con Lean 4

Entonces, ¿cuál es el árbitro infalible que necesitamos?

Lean 4 es un asistente de pruebas con un verificador de tipos que actúa como compilador lógico. Si una prueba tiene un error, **no compila**. No hay ambigüedad. No hay debate.

La diferencia con el lenguaje natural es fundamental. Cuando escribimos "y por lo tanto, combinando ambos lemas, se sigue el resultado", estamos dejando que el lector rellene los gaps con su intuición. En Lean, esos gaps simplemente no existen: cada paso debe justificarse explícitamente, o el verificador rechaza la prueba.

El problema de Lean, y esto es central para entender CoProof, es que **tiene una curva de aprendizaje muy alta**. La mayoría de los matemáticos no saben escribir Lean. Ahí es donde entra nuestra plataforma.

[pausa]

Slide 7 — Prueba Informal vs. Prueba Formal

Este slide ilustra el contraste con un ejemplo concreto: la irracionalidad de $\sqrt{2}$.

La versión en lenguaje natural cabe en cuatro líneas y se entiende en segundos. Pero su validez depende de que el lector acepte cada paso por intuición.

La versión en Lean 4 ocupa veinte líneas y requiere declarar cada hipótesis, cada deducción intermedia, y justificar por qué dos implica par usando un lema de divisibilidad de primos. Es más larga, más técnica... y **matemáticamente incuestionable**.

La pregunta que nos hicimos fue: ¿podemos hacer que generar esa versión formal sea accesible para alguien que solo conoce la versión informal? La respuesta es nuestro módulo NL2FL.

[pausa]

Slide 8 — Lean 4: Corrección Sí, Colaboración No

Entonces Lean es el árbitro perfecto. ¿Por qué no simplemente usar Lean y ya?

Porque Lean resuelve **la corrección**, pero no resuelve **la colaboración**.

Lean es una herramienta de un solo usuario. No tiene infraestructura para distribuir lemas entre colaboradores, no tiene seguimiento de progreso, no tiene gestión de contribuciones, y su curva de aprendizaje hace que la mayoría de los matemáticos no puedan usarlo directamente.

CoProof toma la corrección de Lean y le añade la capa que falta: la plataforma colaborativa. Lean verifica. CoProof coordina.

[pausa]

Slide 9 — CoProof: La Plataforma

CoProof es una plataforma web que unifica cinco componentes en un solo flujo de trabajo.

El **DAG de prueba** organiza el teorema raíz en lemas y casos base. La **verificación automática** pasa cada nodo por Lean 4 antes de aceptarlo. La **colaboración** usa GitHub como fuente de verdad: cada contribución es un pull request que se acepta solo si Lean lo valida.

El módulo **NL2FL** traduce lenguaje natural a Lean usando un modelo de lenguaje con retroalimentación iterativa, para que matemáticos sin experiencia en Lean puedan contribuir. Y el módulo de **cómputo HPC** ejecuta verificaciones computacionales en un clúster MPI, embebiendo la evidencia directamente en la prueba formal.

Todo esto se ilustra con el ejemplo que veremos a continuación: los factoriones.

[pausa]

Slide 10 — Las Tres Operaciones

Cada nodo del DAG admite exactamente tres operaciones.

Split divide un nodo pendiente en sub-lemas independientes. Es la operación de descomposición: permite que diferentes colaboradores tomen partes distintas del trabajo.

Solve demuestra un nodo. El colaborador puede escribir directamente en Lean, o puede escribir en lenguaje natural y dejar que NL2FL lo traduzca. En ambos casos, Lean verifica antes de que la contribución sea aceptada.

Compute añade un nodo de evidencia computacional: lanza un job en el clúster HPC y embebe el resultado como artefacto formal en la prueba. Es la operación que conecta las matemáticas con el cómputo de alto rendimiento.

[pausa]

Slide 11 — Estado del Arte

¿Qué existe ya en este espacio?

LeanDojo y **LeanCopilot**, publicados por Yang et al. en 2023, son el trabajo más cercano al nuestro. Extraen datos de Mathlib4 para entrenar modelos de machine learning que sugieren tácticas en tiempo real dentro del editor. Es excelente para un usuario experto trabajando solo.

El **Archive of Formal Proofs** de Isabelle/HOL es un repositorio comunitario de pruebas formales, pero sin colaboración estructurada ni verificación en tiempo real integrada al flujo de trabajo.

Lo que CoProof agrega es la combinación que ninguno de estos sistemas ofrece: flujo multiusuario con DAG, verificación como requisito de aceptación, traducción de lenguaje natural a Lean, y cómputo HPC como artefacto formal, todo en una sola plataforma integrada.

[pausa]

Slide 12 — Arquitectura

La arquitectura está compuesta por seis capas que se comunican de forma asíncrona.

El **frontend Angular** se comunica con una **API Flask con SocketIO** que mantiene conexión bidireccional en tiempo real. La API sincroniza el estado con **GitHub** como fuente de verdad y persiste metadata en **PostgreSQL**.

Las tareas de verificación se distribuyen vía **Redis** a cuatro workers especializados: el worker de **Lean** para verificación formal, el de **NL2FL** para traducción, el de **Agents** para el orquestador de agentes IA, y el de **Computation** que despacha jobs al clúster de cuatro Raspberry Pi con OpenMPI.

Todo el stack levanta con un único comando: `docker compose up --build`.

[pausa]

Slide 13 — Criterios de Éxito

Presentamos los resultados contra nuestros criterios de éxito definidos al inicio del proyecto.

La tasa de verificación exitosa de Lean alcanzó el **96%** contra la meta del 95%. La latencia promedio de verificación fue de **8.4 segundos**, por debajo del límite de 10 segundos. El stack levanta en un solo comando, el pipeline NL2FL es funcional, la demo end-to-end está completa, y el clúster HPC de cuatro nodos Raspberry Pi está operativo.

El benchmark se realizó sobre cien teoremas de Mathlib4: noventa y seis fueron verificados exitosamente, cuatro presentaron timeout por encima de los 35 segundos, y **cero errores de compilación**. Los resultados validan que la plataforma es funcional y cumple con los objetivos planteados.

[pausa]

Slide 14 — Ejemplo en CoProof: Factoriones

Para ilustrar el sistema de forma concreta, usamos los **factoriones**: números que son iguales a la suma de los factoriales de sus dígitos.

Solo existen cuatro en base diez: el uno, el dos, el ciento cuarenta y cinco, y el cuarenta mil quinientos ochenta y cinco. Verificamos por ejemplo que $1! + 4! + 5! = 1 + 24 + 120 = 145$. Correcto.

La demostración de que no existen más tiene dos partes. Primero, una cota: para n suficientemente grande, la suma de factoriales de dígitos siempre es menor que n —esto se puede mostrar analíticamente—. Segundo, una verificación exhaustiva de todos los números hasta esa cota, que es aproximadamente dos millones y medio. Esa parte computacional es exactamente lo que delega el módulo HPC.

[pausa]

Slide 15 — Factoriones: DAG de Prueba en CoProof

Este es el DAG real del teorema de factoriones tal como queda en CoProof.

El nodo raíz —que los factoriones en base diez son exactamente el conjunto $\{1, 2, 145, 40585\}$ — se divide en dos ramas. La rama derecha, **factorion_upper_bound**, ya está validada en verde: demuestra que no hay factoriones por encima de dos millones y medio. Esa validación se apoya en dos lemas también verificados: la cota de la suma de factoriales de dígitos y el argumento de que n supera esa cota para n grande.

La rama izquierda, **factorion_bounded_exhaustion**, requiere verificar exhaustivamente todos los números hasta la cota. Ese trabajo se delega al **nodo de cómputo HPC** —en naranja—, que lanza el job en el clúster y produce dos resultados: que los cuatro factoriones conocidos sí lo son, y que ningún otro número en ese rango lo es.

Este es el modelo CoProof en acción: demostración formal y cómputo de alto rendimiento como partes del mismo grafo de prueba.

[pausa]

Slide 16 — Demo

Ahora vamos a ver todo esto en funcionamiento en vivo.

El flujo que mostraremos tiene cinco pasos: crear un proyecto con el teorema de factoriones, dividirlo en las ramas que acabamos de ver, resolver un lema en lenguaje natural para que NL2FL lo traduzca a Lean y lo verifique, aprobar el pull request automático que genera el sistema, y finalmente lanzar el nodo computacional al clúster Raspberry Pi.

Quiero cerrar con la idea que abrió esta presentación. La prueba de Atiyah tenía un gap lógico que nadie pudo formalizar con certeza durante meses. Con CoProof, ese gap habría sido **un error de compilador el día uno**.

No eliminamos el trabajo matemático. Lo hacemos verificable, distribuible y transparente. Eso es CoProof.

Gracias. Estamos abiertos a preguntas.

Fin del speech · Duración estimada total: ~11 minutos